universität wien

BACHELOR THESIS

ENHANCEMENT OF THE PROCESS MINING VISUALIZATION TOOL: PETRI NET INTEGRATION, CONVERTERS, TOKEN GAME SIMULATION, AND HAPPY PATH

Author

Jakob Schütz

in partial fulfilment of the requirements for the degree of

Bachelor of Science (BSc)

Vienna, 2026

degree programme code as it appears on the student record sheet:	UA 033 521
degree programme as it appears on the student record sheet:	Informatik
Supervisor:	Dipl.-Ing. Dr.techn. Marian Lux

Abstract

Process mining enables the extraction of process models from event logs and supports the analysis of real-world process behaviour based on real-life data rather than assumptions. Although numerous process mining tools exist, many of them are either commercial solutions that are closed-source and therefore difficult to extend, or open-source frameworks that require expert knowledge and are challenging to use without a strong background in process mining or formal modelling techniques. This thesis contributes to the further development of the Process Mining Visualization Tool, which already addresses this gap between good usability and open-source frameworks.

Open-source process mining tools offer important advantages in this context. First, they enable transparent development. Second, they allow direct implementation and validation of concepts against the literature. At the same time, such tools require continuous improvement in order to remain competitive with mature commercial solutions.

The thesis first introduces the required theoretical foundations, including the genetic algorithm, token game, the concepts of case, trace, and variants, as well as the Happy Path. A special focus is placed on Petri nets and silent transitions as formal mechanisms for modelling routing behaviour that is not given in the event log.
Building on these foundations, existing commercial and open-source process mining tools are reviewed and compared. On the implementation side, the thesis describes the architecture and file structure of the project, provides setup and usage instructions, and documents the implemented improvements.
The main contributions of this work include the integration of a complete Petri net visualization combined with token game semantics, a refactoring of the visualization architecture through the introduction of reusable graph components and a dedicated converter layer, and the implementation of an interactive Happy Path feature. This feature highlights the most frequent trace in an event log and supports the visualization of multiple equally frequent variants.
In summary, this thesis improves the modularity and extensibility of the Process Mining Visualization Tool. In addition, the Happy Path visualization improves the analysis capabilities for event logs.

Contents

1 Motivation

Many organizations manage their daily operations and processes using Enterprise Resource Planning (ERP) platforms, ticketing tools, or workflow engines. However, it is a big challenge to convert this data in intuitive models. In practice, processes often span over multiple business areas, involve multiple exceptions, and deviate from official documentation. Process mining aims to make these problems more tangible by using derived process models from event logs and ultimately making the as-is behaviours of the process visible. It typically consists of three interdependent activities: process discovery, conformance checking, and process enhancement. The practical value of process mining therefore also depends on whether the resulting models and analyses can be visualized in a consistent and intuitive way. This allows stakeholders to understand them, compare different process variants, and identify opportunities for improvement.

Although there are already a lot of different process mining tools, there are still issues concerning accessibility and expandability. Commercial tools usually offer intuitive user interfaces and a wide range of features, but they often require expensive licenses and are usually closed-source, making them difficult to extend. Those factors make them less useful for science, teaching, and community-driven development. Academic tools, on the other hand, often have access to many different mining algorithms and are in many cases open-source. However, they are often more complex to use, which can be a problem for non-experts. To bridge this gap the Process Mining Visualization Tool was developed as an open-source project with a strong focus on usability and extensibility.

Despite the introduction of new features and new mining algorithms, the tool still had potential for improvement. Several core components remained incomplete or were implemented only through temporary solutions. For example, the token game was realized using a workaround rather than a dedicated implementation. Furthermore, the Petri net had not yet been implemented in a reusable manner, limiting its applicability across different parts of the system.

The motivation of this thesis is therefore to enhance the tool and resolve the identified shortcomings of other tools. First, a robust Petri net implementation is developed, which also serves as the foundation for a proper implementation of the token game. After that, the visualization architecture will be refactored, reducing the number of graph types to Business Process Model and Notation (BPMN), Directly-Follows, and Petri net graphs. To further decouple visualization logic from mining algorithms, dedicated converter classes are introduced for each graph type. Finally, a new visualization feature is implemented that allows highlighting the Happy Path which is defined as the most frequently occurring trace within an event log.

2 Related Work

Process mining offers techniques and methods for analysing and visualizing business processes based on event log data. By linking recorded process flow with formal process models, it allows a data-driven understanding of real process behaviour. Specifically, the visualization of discovered models plays a large role in interpreting and facilitating communication among stakeholders, as it helps to reveal dominant behaviour patterns, structural characteristics, and deviations within complex processes. [1]

2.1 Algorithms

2.1.1 Case, Trace and Variants

An event log represents the execution of a process as a set of events. Each event typically contains an activity, a case ID, and a timestamp. A case corresponds to a specific iteration of the process. The trace is a chronologically ordered sequence of events within a case. Formally, a trace can be seen as a sequence of activities, which have been sorted by their timestamps.

Variants can be derived from the set of all traces. A variant is a unique sequence of activities that occurs in multiple cases in the same order. To do this, all traces are projected onto their activity names and identical sequences are grouped. Each group forms a variant with an associated frequency. Instead of looking at thousands of cases individually, one can analyse the few most common behavioural patterns and simultaneously examine rare or deviant processes in detail. Depending on the use case, variants can be formulated strictly or robustly, but the core idea remains to make recurring processes visible as patterns. [2]

2.1.2 Genetic Algorithm

A genetic algorithm is a stochastic optimization method based on the principle of biological evolution. The goal is to find a good, but not necessarily global, solution to an optimization problem in a large, often discrete search space. Unlike classical optimization methods, no single solution is incrementally improved. A genetic algorithm works with a population of candidate solutions, called individuals. Each individual represents a possible solution for the problem. The individual's quality is evaluated using a fitness function that returns a numerical score.

Genetic algorithms represent a general optimization framework that can be applied to a variety of domains. In the field of process mining, these principles are adapted in so-called genetic mining algorithms to automatically discover process models from event logs. In this context, individuals represent concrete candidates for process models, such as Petri nets or similar structures, rather than abstract solutions. The evolutionary mechanisms of the genetic algorithm are adapted to generate valid and meaningful process models and improve them step by step. [3] [4]

2.1.2.1 Process of a Genetic Mining Algorithm

The genetic mining algorithm follows a clearly structured process that is divided into several consecutive steps. Figure 1 shows the main steps of the algorithm, which are explained in more detail below.

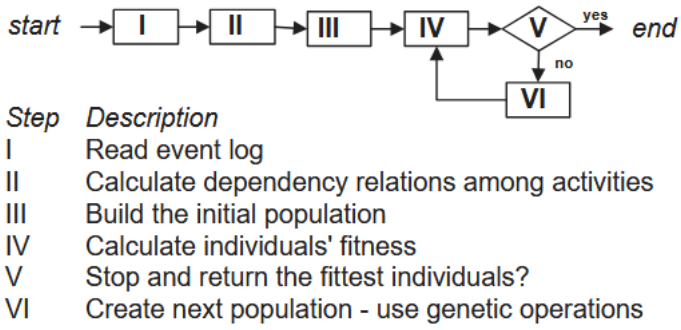


Figure 1: Main steps of the Genetic Algorithm [3]

I. As a first step, the event log is read. This event log forms the basis for the genetic mining algorithm and contains the observed processes in the form of traces.
II. Based on the event log, the dependency relationships between activities are calculated in a second step. This includes analysing which activities typically follow one another and in which direction this relationship exists. This information serves as a structural basis for the later model and is often stored in the form of a dependency or causal matrix.
III. In the third step, the initial population is built. Each individual represents a possible process model based on the previously calculated dependency relations. The initial population can be randomly generated but is usually limited by heuristic information from the event log to avoid obviously incorrect models.
IV. The fourth step involves calculating the fitness of each individual. Each process model is compared to the event log to assess how well it describes the observed behaviour. The fitness function typically measures the extent to which the traces from the log can be reproduced by the model and how much additional, unobserved behaviour the model allows. Models with higher fitness values provide better explanations for the event log.
V. In the fifth step, it is checked whether the termination criterion is met. For example, this can be a maximum number of generations or a time limit.
VI. If the termination criterion is not met, the next population is generated. Genetic operators such as selection, recombination, and mutation are used for this. Fitter individuals are preferentially selected and combined with each other, while mutations intentionally make structural changes to process models to create new variants. The newly formed population then serves as input for the fitness calculation, which starts the evolutionary cycle again. [3]

2.1.2.2 Fitness and Token Game

The token game is based on the formal semantics of Petri nets and related process modelling formalisms. A Petri net consists of places, transitions, and edges. Tokens are placed on places and represent the current state of the process. A transition can be activated if its input conditions are satisfied. More precisely, a transition is enabled if all required input sets are fulfilled (AND semantics across input sets) and within each input set at least one input place contains a token (OR semantics within a set). If that is the case, the transition can be fired which consumes tokens from the selected input places and produces tokens in the corresponding output places. This allows checking whether an observed trace can be executed by the model.

During token replay, a single trace of the event log is processed step by step. It starts with a defined start marker. For every event in the trace, the corresponding transition in the model is selected and fired if it is enabled according to the AND/OR activation rules described above. If a transition cannot be activated, it is interpreted as a discrepancy between the model and the observed behaviour.

After a trace has been fully executed, it is verified whether any tokens remain in the model. Such leftover tokens are an indication that the model still contains open paths or unfulfilled synchronizations after the observed execution. Throughout the replay, the number of consumed tokens, produced tokens, and artificially added tokens is recorded. Based on these quantities, a continuous fitness value is computed that expresses how well the process model fits the trace under consideration. Intuitively, a model achieves a higher fitness if all activities can be successfully parsed according to the AND/OR firing semantics and if no tokens remain after completing the trace. [5] [6]

I. Initialize Petri Net (Start Marking)
II. Select Trace from Event Log
III. Process events sequentially
IV. Check Transition Enablement (AND / OR)
V. Fire Transition
VI. Finish Trace
VII. Compute Fitness Value

The continuous fitness measure is:

$$F(PM, L) = 0.40 \cdot \frac{\text{parsed activities}}{\text{total activities}} + 0.60 \cdot \frac{\text{completed traces}}{\text{total traces}}, \quad F \in [0, 1].$$

[7]

2.2 Visualization

2.2.1 Petri Net

A Petri net is a directed graph consisting of places and transitions as can be seen in Figure 2 with its corresponding log data in Figure 3. Places can contain tokens, and transitions fire as soon as their input places have enough tokens. After completing the corresponding mining algorithm, the best candidate model is transformed into a Petri net to make it analysable and visualizable. A workflow-like structure with an explicit start and end is typical. A start node and an end node are connected via dedicated places, so it is clear where tokens are initially generated and where they are collected at the end. The input and output relationships contained in the model are then modelled using intermediate places. [6]

- For each input set of an activity, a Place is created that connects the predecessors with the activity.
- For each output set, a place is created that connects the activity with its successors.
- Self-loops are treated separately by introducing a place that links the activity to itself.

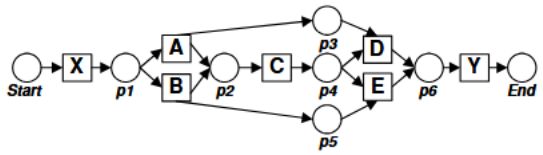


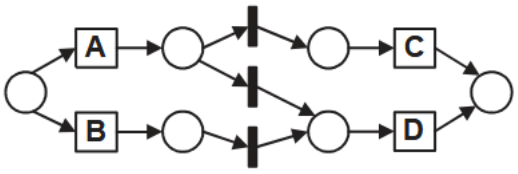
Figure 2: Petri net model [5]

ID	Process instance	ID	Process instance	ID	Process instance
1	X, A, C, D, Y	3	X, A, C, D, Y	5	X, B, C, E, Y
2	X, B, C, E, Y	4	X, B, C, E, Y	6	X, A, C, D, Y

Figure 3: Corresponding event log [5]

2.2.2 Silent Transitions

When constructing a Petri net from a causal matrix, silent transitions are commonly introduced in the literature [5]. These transitions have no counterpart in the event log and are mainly used to correctly represent control flow constructs, such as choices, parallel splits/joins, and certain loop structures, as can be seen in Figure 4 with its corresponding causal matrix in Figure 5. [5]



(a) Naive translation.

Figure 4: Petri net with silent transitions [3]

$ACTIVITY$	$INPUT$	$OUTPUT$
A	$\{\}$	$\{\{C, D\}\}$
B	$\{\}$	$\{\{D\}\}$
C	$\{\{A\}\}$	$\{\}$
D	$\{\{A, B\}\}$	$\{\}$

Figure 5: Causal Matrix [3]

2.2.3 Happy Path

The Happy Path refers to the typical or desired sequence of events in the process. In a log-based model, the Happy Path is often operationalized as the most common variant. When considering the model from a normative perspective, the Happy Path can also be determined by domain knowledge. In this case, the Happy Path can deviate from the most frequent trace. In both cases, however, the Happy Path represents an anchor point for interpretation, communication, and evaluation. Deviations can also be described and quantified, and in a further step, their causes can be discussed.

To support the identification and interpretation of the Happy Path in process models, visual encoding techniques such as colour-based clustering or the explicit highlighting of relevant edges and activities can be employed (see Figure 6). [8] [9]

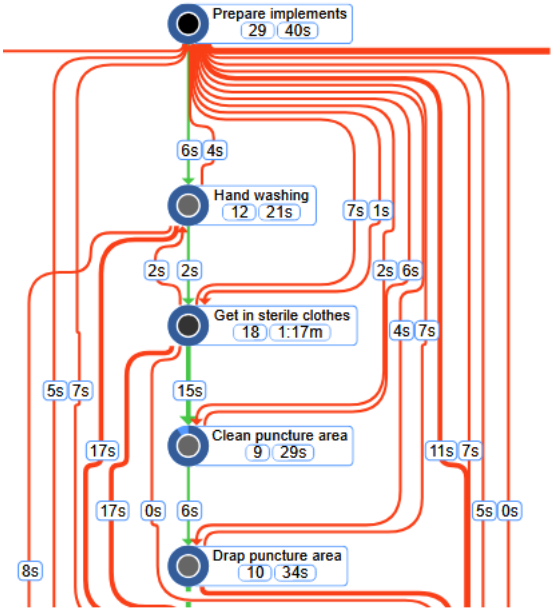


Figure 6: Happy Path with green highlighting [10]

2.3 Existing Tools

In the area of process mining, a wide range of tools exists that differ with respect to functionality, licensing models, and target groups. These tools can broadly be categorized into commercial and open-source solutions. The following section presents this classification by introducing the most

prominent representatives of each category and subsequently discussing the main limitations of those tools.

2.3.1 Commercial Tools

Commercial process mining tools primarily target business users and offer a high degree of integration with existing IT systems as well as advanced analytical capabilities. A prominent example is Celonis, a mainly cloud-based platform that is mainly used in large organizations. The tool provides the main functionalities such as process discovery, conformance checking, and the enhancement of existing process models. Additionally, Celonis supports a wide range of data mining algorithms, including the Heuristic Miner, the Inductive Miner, and object-oriented process mining approaches. In addition to typical analysis functions, the platform offers customizable dashboards which are tailored to different organizational roles, such as management or specialized departments. Despite this wide range of functions, the applicability of Celonis is limited for smaller organizations and research institutions due to high license and operating costs. [11]

Another widely used commercial process mining tool is Disco. Disco is characterized by its ease of use and the ability to deliver quick insights during the exploratory analysis of event logs. The application operates locally as a desktop solution and requires manual data preparation as well as manual data import, typically in CSV format. Unlike typical company-oriented platforms, Disco only focuses on a few analytical functions and supports only one mining algorithm, the Fuzzy Miner. The lack of support for additional algorithms combined with the proprietary architecture significantly limits the tool's adaptability and restricts its usability in advanced or experimental scenarios. [12]

2.3.2 Open-Source Tools

Unlike many commercial solutions, ProM 6 represents an open-source approach. Its modular framework provides a large number of plugins for different process mining tasks, including process discovery, conformance checking, and performance analysis. The openness of the platform enables researchers to also implement their own custom algorithms and modify the existing ones. However, the platform is noticeably more complex to use and requires some background knowledge of the underlying concepts and process mining principles. Furthermore, ProM is less suitable for industrial environments, since aspects such as scalability, the user experience, and integration with operational information systems are only partially addressed. [13]

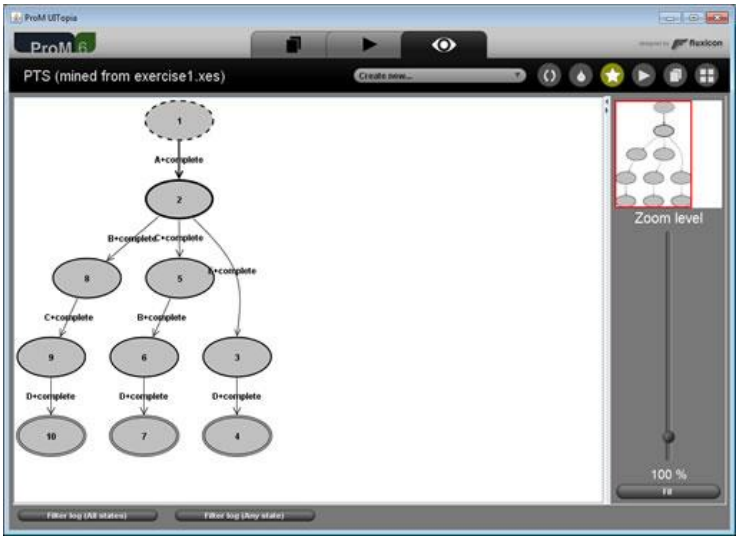


Figure 7: ProM 6 User interface [13]

2.3.3 Critical Review of Existing Tools

Despite the wide variety of available process mining tools, several fundamental weaknesses persist. Commercial solutions typically offer extensive functionality and strong integration with existing IT infrastructures. However, they are often associated with high costs and only limited possibilities for customization. Proprietary algorithms and closed-source systems decrease the transparency and make it difficult to reproduce analysis results. On the other hand, open-source tools such as ProM 6 are characterized by a high degree of flexibility and transparency but are also connected with a steep learning curve and rather complex user interfaces. This comparison shows a clear gap between intuitive but restrictive commercial solutions and flexible but complex to use research-oriented tools, highlighting the need for a tool that combines usability with openness and extensibility. [14]

3 Implementation

This chapter documents the technical implementation of the application. At first, the overall architecture of the system is introduced. Then, the project setup and the usage of the application are described, including the file structure. Afterward, the implemented improvements are presented. Finally, practical aspects such as testing and the most important possibilities for extending the project are discussed. The full source code of the implementation is openly available at: https://github.com/jakob655/ProcessMiningVisualization_Schuetz_WS2025

3.1 Architecture

The project was implemented as a modular Streamlit [15] tool and follows the Model-View-Controller (MVC) principle. The goal of this structure is to create a clear separation between data storage and mining logic, user interface, and controller, which coordinates both. This design decision was already made in previous developments of the tool, but it was further maintained in this thesis.

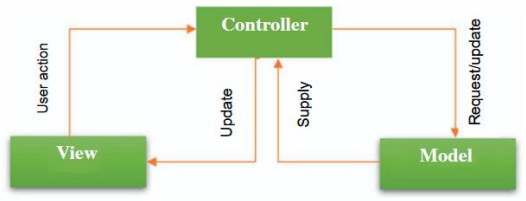


Figure 8: MVC model [16]

- **Model**
  The model component includes data preparation, mining algorithms, and the generation of process models. Imported CSV files are initially processed as a Data Frame and then converted into an internal log representation (traces). The actual discovery algorithms are encapsulated as mining models and follow a common interface. Key functions such as log preprocessing, frequency calculations, and standard filters are bundled in a common base class. This ensures that all algorithms have common behaviour regarding filtering and statistical calculation, while their algorithm-specific steps remain implemented separately. As a result, the model then generates not only data, but also a graph object (e.g., Petri net or DFG) that provides a Graphviz [17] representation as well as metadata for nodes and edges (e.g. frequencies).

- **View**
  The view layer was implemented with Streamlit. Each page has its own view class that renders its UI elements (table views, buttons, sliders) and structures output components, such as graphs. The algorithm page uses a common base view that encapsulates recurring UI patterns: A sidebar for filter options, a central container for the graph, and a separate area below it where further details about nodes or edges can be displayed with a click.

  A separate interactive Streamlit component is used for graph rendering. This enables interactions such as “click on activity -> show more information” independently of the mining algorithm.

- **Controller**
  The controllers connect user interactions with model functions. Each page has its own controller class, while algorithm pages use a common base controller that implements recurring processes such as updating the display or starting the mining process.

  The controller is thus the central location for flow and error logic (e.g., invalid file types, missing column selection) and ensures that the view remains as free as possible from business logic.

3.2 File Structure

The code base is divided into independent modules according to responsibilities and reflects the MVC architecture directly in the folder structure. The central entry point is *streamlit_app.py*, which sets the Streamlit configuration and routes between pages using *st.session_state*.

The most important directories are:

- ***ui/***
  Contains the Streamlit pages. Each page is in its own subfolder (e.g., *home_ui* or *column_selection_ui*). These folders typically contain a controller and a view class. Common base classes are located in *ui/base_ui/.*

- ***mining_algorithms/***
  Contains the implementation of mining algorithms (Alpha, Heuristic, Inductive, Fuzzy and Genetic Miner) as well as shared infrastructure (*mining_interface.py, base_mining.py*). These base classes encapsulate reusable steps such as log preprocessing and standard filters.

- ***graphs/***
  Contains process models and help structures as well as graph classes for representation, based on Graphviz (e.g., *base_graph.py, directly_follows_graph.py*).

- ***converters/***
  Contains classes with methods which convert algorithm-specific results into displayable graphs, such as converting the result of the Genetic Miner into a Petri net.

- ***transformations/***
  Transformations related to Data Frames and logs (including grouping by case and sorting by time), as well as styling aids for the column selection view.
- ***io_operations/***
  Import/Export logic: CSV import, reading files, and exporting graphs.
- ***tests/***
  Unit tests and test data, structured by module (e.g., mining algorithms and graphs).
- ***docs/***
  contains documentation (including algorithm descriptions, diagrams, and previous theses), as well as templates for integrating new UI pages or mining algorithms into the existing structure.

3.3 Setup and Application Usage

The tool was implemented in Python and uses well-known libraries such as NumPy [18], Graphviz [17], and Streamlit [15]. These and other dependencies can be installed using the following command:

```
pip install -r requirements.txt
```

After that, the application can be easily started from the root directory with the following command:

```
streamlit run streamlit_app.py
```

3.3.1 Home / Import

First, a file must be uploaded on the Home page. The following file types are supported:

- **CSV event logs** for mining new process models.
- **Pickle files** for loading already exported models.

For CSV files, the tool attempts to recognize the delimiter from the first line and adopts it as a suggestion. Figure 9 shows the page.

Figure 9: Home Page

3.3.2 Column Selection Page

After the CSV file has been uploaded, the required columns are mapped to the internal log representation. Three columns are required for standard discovery:

- Timestamp
- Case/Instance-ID
- Event

The required columns are automatically detected by the system. If this automatic detection fails, the user can manually select the appropriate columns. Afterwards, the user can select a process mining algorithm from a drop-down menu. After choosing the desired algorithm, the mining procedure is started by clicking the “Mine” button. Figure 10 shows the page.

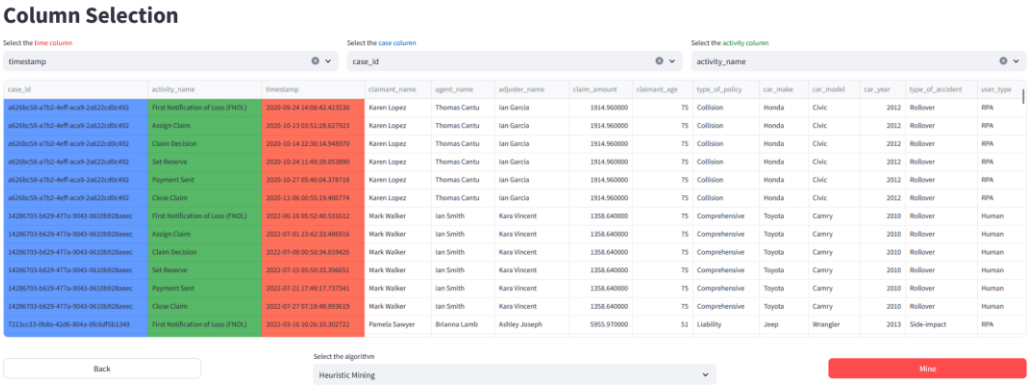


Figure 10: Column selection page

3.3.3 Mining Algorithm Page

After selecting the desired algorithm on the Column Selection page, the corresponding algorithm page opens. All algorithm pages follow a common layout:

- Sidebar with filter and algorithm parameters
- Central graph
- Detail area for node and edge information

Some common filters, such as node frequency thresholds, are available for all algorithms in the sidebar and can be adjusted interactively. If a parameter is changed, the model is recalculated and the graph is updated. In addition, there is a Happy Path option that highlights the activities of the most frequently occurring trace in the currently filtered log. If several traces have the same maximum frequency, the desired variant can be selected. Figure 11 shows the page.

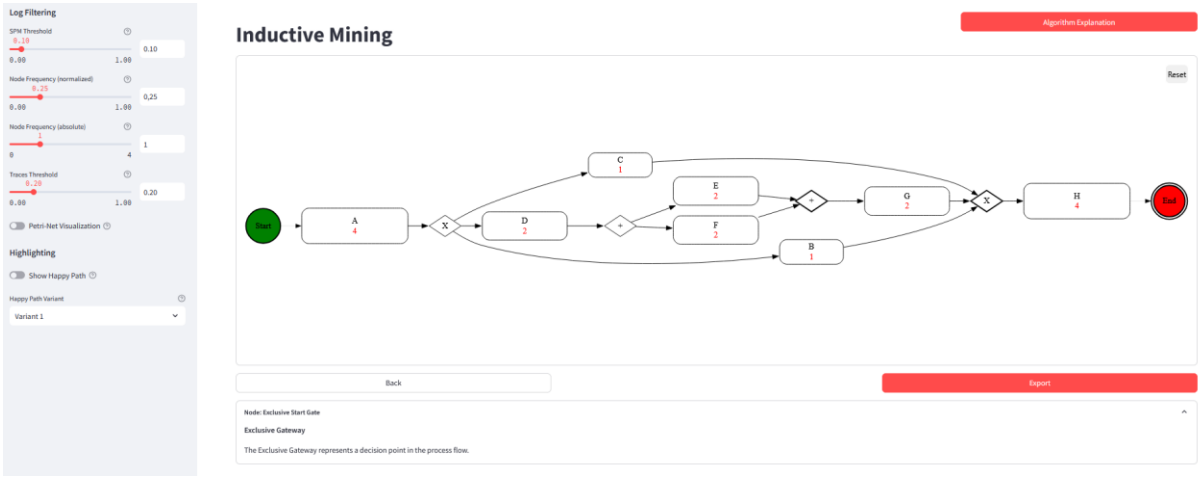


Figure 11: Mining algorithm page

3.3.4 Export Page

Once a model has been generated, it can be exported via the dedicated export page. The graph is then exported as either SVG, PNG, or DOT using Graphviz. There is also the option to export the model as a pickle so that it can be loaded in a later step with the same parameters to be further analysed. Figure 12 shows the page.

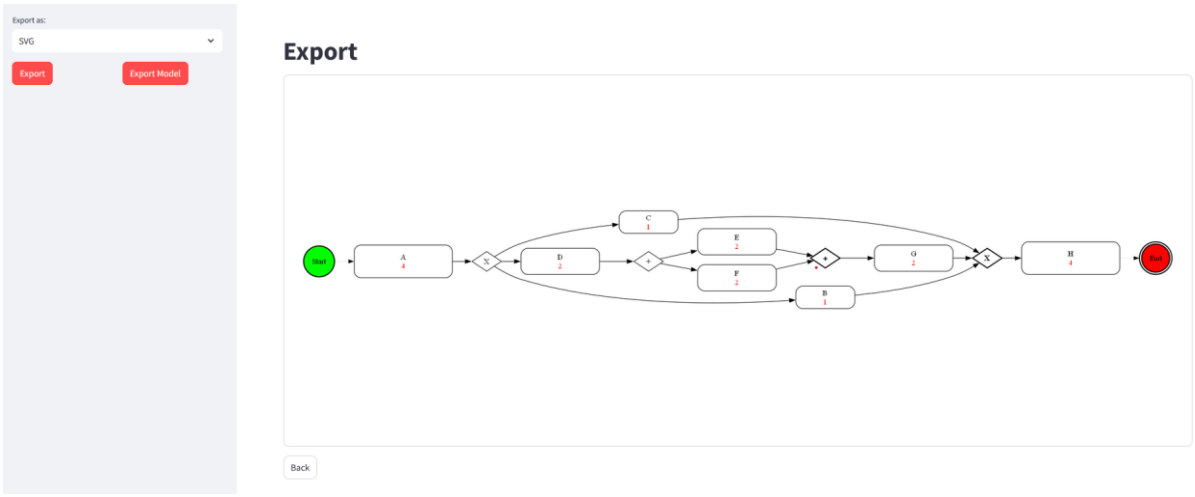


Figure 12: Export page

3.4 Implemented Improvements

In the course of this thesis, the Process Mining Visualization Tool was functionally expanded and structurally refactored. The focus was primarily on implementing a reusable Petri net logic, a token game including silent transitions, a more uniform visualization of the graphs, and the introduction of a converter layer to separate the graph generation logic from the algorithm logic. In addition, minor improvements were made to the UI and the option for visualizing the Happy Path was implemented. The following points describe these key changes.

3.4.1 Petri Net and Token Game

The first step was to implement the Petri net core data structure. In practice this meant implementing containers such as a places-set, a transitions mapping (holding input and output sets), an arcs-set for the connections, and dictionaries for initial markings and final places. Based on this core structure, an initial Petri net builder was created, which in the first version generated only visible transitions and integrated them into the net. In practical terms, this meant that one transition per activity in the log was created and linked to each other and to the boundary places. In the code, this meant calling the implemented helper methods: *register_transition(…)* and *add_arc(…).* The next step was to expand the builder by integrating input and output places. Instead of connecting the transitions directly to each other, intermediate places were used. These intermediate places represent join and split structures more precisely. Concretely, this required implementing unique place IDs, maintaining mappings (e.g., which input place belongs to which activity/subset), and ensuring that the arcs are created in the correct direction. In a further step, silent transitions were integrated into the builder, allowing certain scenarios such as loops or synchronizations to be modelled more precisely. From an implementation perspective, this was mainly a routing mechanism. Instead of directly connecting certain places to other places, the code inserts an additional transition node with *visible=False* and connects it via arcs (place → silent → place).

After that, the Petri net structure was linked with a token game logic. The core method here is the *_fire(…)* method (see Figure 13). It updates the marking by decrementing tokens in all input places of the fired transition and incrementing tokens in all output places. In code, this method is intentionally kept small because it is called frequently during replay. The correctness of the overall token game therefore depends on two things: The builder must populate transition's input/output sets correctly, and *_fire(…)* must update the markings consistently for every firing step. To make the token game robust for real logs and generated nets, additional helper methods were implemented. These include methods for firing forced silent transitions (*_fire_silent(…)*) and a recursive *ensure_token* method that makes tokens available along permitted paths when the next transitions would otherwise be blocked. These changes were initially implemented in the Genetic Miner's algorithm class but were then transferred to a separate class (*PetriNetToolkit*). This also enabled other mining algorithms to visualize their model using a Petri net. This was applied to the following Algorithms:

- **Alpha Miner:** The visualization has been completely converted to the Petri net structure.
- **Inductive Miner:** In addition to the existing BPMN visualization, the current visualization was supplemented with a Petri net visualization. This allows users to switch between BPMN and Petri net visualization in the UI.
- **Genetic Miner:** The visualization is based entirely on the Petri net structure.

```python
@staticmethod
def _fire(transitions: dict, marking: dict, transition_id: str) -> None:
    """
    Fire transition:
    remove/add token
    """
    transition = transitions[transition_id]

    # consume from input places
    for place in transition['inputs']:
        marking[place] = marking.get(place, 0) - 1

    # produce for output places
    for place in transition['outputs']:
        marking[place] = marking.get(place, 0) + 1
```

Figure 13: Method for firing tokens

3.4.2 Refactoring of the Visualization Architecture

A crucial step was separating the mining logic from the visualization logic. Instead of separate visualization classes for the individual algorithms, a common graph basis was established that can be reused by multiple miners. To achieve this, the logic of the three central graphs (Directly-Follows

graph, BPMN graph, and Petri net graph) was consolidated into three dedicated graph classes. Each of these classes encapsulates the graph-specific methods and implements a common interface (*BaseGraph*). This enables interactive features such as exporting models to continue functioning independently of the selected graph. Secondly, a converter layer was implemented to generate graphs from the results of the mining algorithms. Instead of constructing Graph structures directly inside the mining classes, each algorithm now prepares its algorithm-specific results and delegates the visualization step to a converter. Therefore, each algorithm passes its required input via dedicated data classes (see Figure 14). This provides a clear contract between mining algorithms and converters and keeps algorithm-specific data structured and consistent. The mining algorithms were therefore also adapted to follow a consistent pipeline. First, they build the mining-specific model. Then they assemble the required data class and afterwards call the converter to build the final graph.

This change has two main advantages: First, it makes the mining implementations more coherent because they focus on mining logic and reuse shared visualization infrastructure rather than duplicating code. Second, the visualization is implemented centrally in one place, which means that the Graphs remain consistent across multiple miners and extensions do not have to be implemented multiple times.

```python
@dataclass
class AlphaPetriNetData:
    nodes_to_draw: set[str]
    start_nodes: set[str]
    end_nodes: set[str]
    yl_set: set[tuple]
    filtered_events: set[str]
    filtered_appearance_freqs: dict[str, int]
    node_stats_map: dict[str, dict]
    edge_filter: Callable[[str, str], bool] | None = None
```

Figure 14: Converter data class

3.4.3 Happy Path & Variants

The visualization of the Happy Path was implemented as a user-oriented enhancement. The implementation is based on the definition of the Happy Path as the most frequent trace in the filtered log. From an implementation perspective, the feature is split into three clearly separated steps to keep it reusable across different mining algorithms and graph types.

First, the most frequent trace is determined from the filtered log. This is implemented in *base_mining*. Since the log is stored as a dictionary mapping traces to their frequencies, the method *get_happy_path_traces()* determines the maximum trace frequency and collects all traces

that match this frequency. The traces are then sorted by their length and are returned afterwards. The method *get_happy_path_trace(…)* then selects either the default trace or a specific variant.

Second, the UI integrates the feature with a simple toggle on the algorithms page. When activated, the controller collects the Happy Path activities from the mining model and forwards them to the graph instance.

Third, the implementation has been extended to allow the visualization of multiple equally frequent variants. If the model exposes multiple equally frequent traces, the view renders an additional drop-down menu, allowing the user to select one of the variants. The selected index is then stored in *st.session_state*.

Highlighting was intentionally implemented as a pure visualization function in *base_graph*. Activities are highlighted without re-executing the mining steps. This keeps interaction fast and allows users to directly identify the typical flow in the model. However, it should be noted that this is only a simplified highlighting of the Happy Path. Highlighting the full path, would require additionally marking places and edges, which depends on algorithm-specific routing semantics and was therefore not implemented within the scope of this work.

```python
def get_happy_path_traces(self) -> list[tuple[str, ...]]:
    """Return all most frequent traces in the current filtered log.

    """
    source_log = self.node_frequency_filtered_log
    if not source_log:
        return []

    max_frequency = max(source_log.values())
    most_frequent_traces = [trace for trace, frequency in source_log.items() if frequency == max_frequency]
    if not most_frequent_traces:
        return []

    return sorted(most_frequent_traces, key=len, reverse=True)
```

Figure 12: Method to get the most frequent traces

3.5 Testing

Testing is an important step in software development. Different testing strategies are used to ensure that software works correctly and without errors. As part of this work, both Unit tests and manual checks were used to ensure correctness despite major refactoring.

During the implementation phase, several structural changes were made, particularly to the Petri net implementation and the token game logic. These components are closely linked and therefore changes to the net structure have a direct impact on the parsing and replay behaviour of the token game. For this reason, existing unit tests for the Genetic Miner were no longer directly compatible after the refactoring and had to be adapted. For the changes regarding the visualization architecture, the existing unit tests were reused.

The Happy Path visualization, on the other hand, is mainly UI-driven. For this purpose, the functionality was verified manually. To do this, several small CSV logs were created covering different

scenarios. The running tool was then used to check whether the calculated Happy Path traces were correctly recognized and whether the expected activities were consistently highlighted in the graph. By combining customized unit tests for the Petri net and token game changes and manual UI validation for the Happy Path feature, the functionality of the application could be reliably ensured even after extensive structural adjustments.

3.6 Extending the Project

An important feature of this project is that it can be easily expanded and adapted to one's needs, which is why it was intentionally designed as an open-source project. This is based on the consistent use of the MVC architecture and common base classes. Extensions are therefore typically made along clearly defined entry points.

3.6.1 Adding a New Page

New pages are created in the *ui/* folder as a separate subpackage. This usually requires two new classes:

- **Controller**
  Inherits from *base_controller.py* and is responsible for navigation, session state management, input validation, and triggering model functions.
- **View**
  Inherits from *base_view.py* and contains only visualization logic.

The page is then integrated into the navigation via *streamlit_app.py*, together with a route.

3.6.2 Adding a New View to a Page

If a page requires multiple display variants (e.g., different layouts or special detail views), multiple view classes can be stored in the same UI subpackage. The controller is then initialized with a list of views and selects one in *select_view(),* typically based on *st.session_state* values. To avoid duplication, it is recommended to start from the provided page template in *templates/ui_template*. The new view then defines the new requirements and exposes the required keys so the controller can store them in *st.session_state*. If the additional columns must be included in the internal log representation, the corresponding algorithm controller overrides *transform_df_to_log()* to pass the extra attributes.

3.6.3 Adding a New Column Selection

It may happen that a new algorithm requires additional attributes besides case, timestamp, and activity. The column selection page must be extended for this purpose. A new column selection view can be created based on the two templates in *templates/column_selection*.

3.6.4 Adding a New Mining Algorithm

Adding a new mining algorithm essentially requires three components that work together.

1. **Mining Model**
   The implementation of the actual algorithm is located in *mining_algorithms/* and should ideally inherit from *BaseMining*.

2. **Graph/Converter-Integration**
   Instead of creating the Graphviz structures directly in the algorithm class, new models are integrated via the converter layer. Depending on the desired graph, an existing converter is used and extended with a new method. The algorithm specific data can be implemented in a separate data class in the converter.

3. **UI-Integration**
   For the user interface, a new subfolder must be created in the *ui/* folder based on the template in *templates/algorithm_ui_template/.*
   - The algorithm controller inherits from *base_algorithm_controller.py*. Its main task is to read the parameters from Streamlit's *session_state*, start the mining process, and rebuild the graph when parameters change.
   - The Algorithm View inherits from *base_algorithm_view* and only adds the algorithm-specific parameters and control elements to the sidebar.

Finally, the new algorithm is registered in *config.py*.

3.6.5 Adding a New Graph

If a new graph type is required, it is advisable to add a separate graph class to *graphs/visualization/*. This class should then inherit from *BaseGraph* in order to implement the necessary functionality. To separate the visualization from the mining logic, the new graph should be generated using the converter layer. To do this, a new converter class including a suitable method must be implemented. The required input data can be collected and transferred to the converter using a new data class. In a final step, the desired mining algorithm is adjusted. Since all algorithms already render *BaseGraph* objects with the common *BaseAlgorithmView*, no changes to the UI are necessary.

4 Evaluation & Discussion

This chapter discusses the main challenges encountered during the development of this thesis, improvements to the tool and concludes by outlining the current limitations of the project as well as summarizing what has been learned during the development.

4.1 Challenges

The first major challenge was not the implementation of individual features or methods but rather understanding and familiarization with an already grown code base. The project already consists of several mining algorithms, different graph types, and a Streamlit-based UI. Since my prior knowledge in some of these areas was limited, I needed some time to get an overview. Although this first phase was time-consuming, it also laid the foundation for implementing structural changes such as the converter layer or the Petri net into the overall project.

Another challenge was the implementation of the token game. The intended execution semantics, especially with places, silent transitions, and cases such as XOR and loop behaviour, are easy to misinterpret if you only look at the code. To avoid implementation errors, it was even more important to refer to suitable literature to ensure semantically correct behaviour.

4.2 Evaluation Approach

To determine whether the implementation is correct, a combination of unit tests, comparison with literature and manual validation were used. First, the refactoring of the Petri net and the token game was verified. For this purpose, the existing unit tests had to be adjusted to the new Petri net structure and semantics of the token game. For example, the tests were adjusted so that only a single firing was allowed at the beginning of the replay. In addition, the expected behaviour of the token game, especially regarding enablement, firing order and the handling of silent transitions, was validated against appropriate literature. Second, the existing unit tests for the other mining algorithms were reused as a baseline for the changes to the visualization architecture. This was particularly important when implementing the converter layer, as errors in this area can affect several mining algorithms at once. Therefore, the existing tests that cover core functionality were used to ensure that unrelated behaviour remained stable.

Finally small manual tests were performed to evaluate UI-driven features. For example, several CSV files representing different situations were created to verify the Happy Path functionality. Since the expected Happy Path traces were predefined, it was possible to verify if the detected variants aligned with literature and if the highlighted nodes corresponded to the selected trace.

4.3 Limitations

One significant limitation concerns the current Happy Path visualization. In the current implementation, the Happy Path is determined based on the most frequent trace in the filtered log, and only the activities in the resulting model are highlighted. This solution is functional and allows the Happy Path to be recognized, but it does not consistently map the complete path in the Graph. For complete marking, edges, places, and silent transitions would also have to be taken into account. This is where a problem arises: Which places and edges belong to the Happy Path cannot be derived from the graph structure alone but depends on the specific model construction and routing decisions.

This lack of knowledge is particularly evident in XOR and AND constructs, as can be seen in Figure 16. In XOR structures, a decision would have to be made as to which branch corresponds to the Happy Path and in AND structures, the question arises as to which parallel paths should be marked as part of a Happy Path. A consistent solution therefore requires algorithm-specific logic that reconstructs the Happy Path not only as a set of activities, but as a concrete token flow or path in the model. This cannot be solved purely at the visualization level but would have to be implemented at the algorithm or converter level. Such an extension would go well beyond the scope of this work and was therefore intentionally not implemented.

In addition, there are limitations due to Streamlit. Streamlit is very well suited for fast and easy interaction but offers only limited possibilities for complex graph visualizations. Even though the Happy Path visualization could be further improved conceptually (e.g., by using a frame-like or path-based highlighting along the entire flow), such a representation cannot be implemented in the current Streamlit setup without additional, extensive front-end effort.

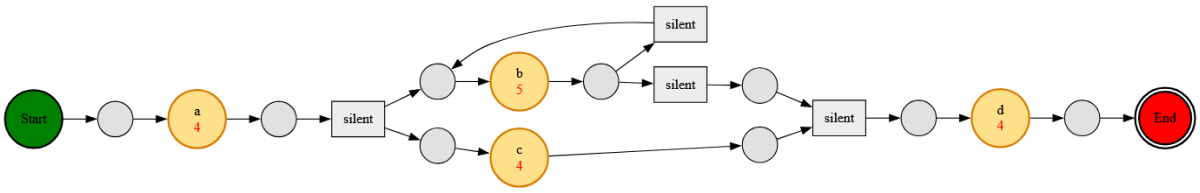


Figure 16: Ambiguity in Happy Path highlighting

4.4 Lessons Learned

The most important lesson I learned during this bachelor thesis is that working on an existing project begins with understanding, not implementation. The greatest effort was initially spent on understanding the architecture, data flow, and responsibilities. Only after this first step it was possible to make changes without negatively affecting other parts of the tool.

Another important point I will take away from the project is the importance of good extensibility. In a project that is developed over several semesters and by different people, a clear structure determines how quickly new features can be integrated. Shared base classes, uniform interfaces, and a clean separation of mining logic and visualization reduce future effort.

5 Conclusion & Future Work

5.1 Conclusion

The goal of this thesis was to expand the Process Mining Visualization Tool and improve its maintainability and extensibility. Limitations of current tools were addressed: Commercial tools are often closed-source and difficult to expand, while open-source tools often require a lot of prior knowledge and are therefore often not usable for new users. By structuring and improving the internal architecture, the project is bridging this exact gap in existing process mining tools. The central output of this thesis is therefore a clear separation between mining logic and visualization. Instead of algorithm-specific graph creation, reusable graph classes and a converter layer were implemented, allowing different miners to create consistent graphs. This not only prevents duplicate code but also means that changes only need to be made in one central location. In addition, a correct Petri net with an associated token game was implemented. And finally, the tool was expanded with a Happy Path visualization feature, which even allows different variants to be displayed.

The results indicate that the refactoring was successfully integrated into the project and that the Petri net, including silent transitions and token game, is consistent with the theoretical basis. At the same time, however, there are still minor limitations, especially regarding the Happy Path highlighting.

5.2 Future Work

Some points can still be addressed to further improve the Process Mining Visualization Tool:

- **Happy Path Highlighting**
  In the current implementation only the corresponding activities are highlighted. A more complete approach would be to reconstruct the Happy Path as a concrete flow through edges, places and silent transitions. To achieve this, some kind of mapping logic in the algorithm or converter logic is needed. That way especially XOR and AND constructs will be easier to analyse in the Happy Path.

- **Algorithm Expansion**
  Due to the tool's modular architecture, new discovery algorithms can be integrated with rather low effort. Adding additional miners would broaden the range of supported use cases and therefore make the tool relevant for a larger target group in teaching, research, and practical exploratory analysis.

- **Design Improvements**
  While the tool already has a functional UI design, it is still important to continuously improve it. This is especially important in order to be appealing to new users. Exploring different design approaches and gathering user feedback can help to develop an even more visually appealing and intuitive tool. A clean and consistent design is not only aesthetic, but it also has an impact on how quickly users understand the interface.

- **Improving Usability**
  Another minor point that could be improved is that when changing the mining algorithm, it should not be necessary to upload the same file again. It would be helpful and more convenient if you could go back directly to the Column Selection Page when you want to use a different algorithm.

6 Bibliography

[1] W. M. P. van der Aalst, Process Mining: A 360 Degree Overview, Springer, 2022.

[2] W. M. P. van der Aalst, Process Mining: Data Science in Action, Springer-Verlag Berlin Heidelberg, 2016.

[3] A. K. A. de Medeiros, A. J. Weijters and W. M. P. van der Aalst, “Using genetic algorithms to mine process models: representation, operators and results,” Eindhoven University of Technology, 2004.

[4] A. K. A. de Medeiros, A. J. Weijters and W. M. P. van der Aalst, “Genetic process mining: an experimental evaluation,” *Data Mining and Knowledge Discovery,* vol. 14, no. 3, pp. 245-304, 2007.

[5] W. M. P. van der Aalst, A. J. Weijters and A. K. A. de Medeiros, “Genetic Process Mining: A Basic Approach and its Challenges,” Eindhoven University of Technology, 2005.

[6] T. Murata, “Petri Nets: Properties, Analysis and Applications,” *Proceedings of the IEEE,* vol. 77, no. 4, p. 541–580, 1989.

[7] D. Kapfhammer, Enhancement of the Process Mining Visualization Tool: Merge Projects, add Filtering, UI-Enhancements and Genetic Miner Implementation, 2025.

[8] M. Lux and S. Rinderle-Ma, “DDCAL: Evenly Distributing Data into Low Variance Clusters Based on Iterative Feature Scaling,” *Journal of Classification,* p. 106–144, 2023.

[9] W. M. P. van der Aalst, “On the Pareto Principle in Process Mining, Task Mining, and Robotic Process Automation,” 2020.

[10] J. Meyer, J. Reimold and C. Wehmschulte, “Conformance Checking Challenge 2019: Analysis of Central Venous Catheter Installation with MEHRWERK ProcessMining,” 2019.

[11] Celonis, “Process intelligence and process mining,” [Online]. Available: https://www.celonis.com/. [Accessed 06 01 2026].

[12] Fluxicon, “Process mining and automated process discovery software for professionals,” [Online]. Available: https://fluxicon.com/disco/. [Accessed 06 01 2026].

[13] “Prom tools,” [Online]. Available: https://promtools.org/. [Accessed 06 01 2026].

[14] W. M. P. van der Aalst, Process Mining Discovery, Conformance and Enhancement of Business Processes, Springer Verlag Berlin Heidelberg, 2011.

[15] Streamlit Inc., [Online]. Available: https://streamlit.io/. [Accessed 10 01 2026].

[16] M. Ehsan Rana and S. S. Omar, “High assurance software architecture and design,” in *System Assurances Modeling and Management*, Academic Press, 2022, p. 271–285.

[17] Graphviz, [Online]. Available: https://graphviz.org/. [Accessed 10 01 2026].

[18] NumPy, [Online]. Available: https://numpy.org/. [Accessed 10 01 2026].